

AI Voice App — Audit & Optimization Report

InterviewCopilot · live-call listening assistant · backend + desktop overlay

Repository	c:\uday mannlowe\ai-call
Commit audited	7a375cf "foundation" · 2026-08-23 12:47 +0530 · working tree clean
Date	23 August 2026
Phase	Audit only. No application code, configuration, prompt, model or threshold was modified. All findings below are measurements and recommendations.
Evidence base	10,130 logged production events across 205 session files (7 days) · 33-stream scored VAD corpus · 1,535 unique real transcripts replayed through the gate · live Groq STT/LLM benchmarks with rate-limit headers · 40 judged answer-quality samples · offline end-to-end pipeline runs

Evidence classes used throughout — never mixed:

VERIFIED confirmed by code inspection or a deterministic test · **OBSERVED** seen in runtime measurement or production logs · **INFERRED** strongly supported by evidence, not directly confirmed · **RECOMMENDED** proposed future change · **REQUIRES VALIDATION** must be tested before implementing · **N/A** stage does not exist in this application

1 · Executive Summary

This application is **substantially better engineered than its symptoms suggest**. The segmentation, barge-in, cancellation and error-recovery logic are careful, well-reasoned and — where I could test them — correct. Its problems are concentrated in three specific places, and two of them are configuration-shaped rather than architectural.

2.9 questions/min before the LLM provider throttles (measured from live rate-limit headers)	50% of noise-only audio streams that still trigger a transcription request	-36dBFS speech quieter than this is never detected at all on a quiet line	1.02s VAD end-of-speech → first visible token, healthy path
--	--	--	--

The three things that matter

P0 — The system prompt starves the app of its own rate limit

The answer system prompt is **1,978 tokens** and is re-sent on every question. Against the account's measured limit of **8,000 tokens/minute** for `openai/gpt-oss-120b`, one question costs **2,719 tokens** — so the app throttles after **~3 questions per minute**. A real interview exceeds that. Past the limit answers do not fail loudly, they *stall*: measured time-to-first-token of 8.1s, 15.3s and 17.5s against ~0.5s unthrottled, plus outright **429 s** in the logs. This single fact explains the `p99 = 17.5s` and `max = 126s` tails in the production data.

P1 — "Sound not detected" has two distinct root causes, both in the energy VAD

(a) Deafness on a quiet line. `VAD_ENERGY_THRESHOLD=500` is an absolute floor the adaptive logic can never go below, so speech below 500 RMS (-36 dBFS) produces no segment, no STT call, no transcript and *no log line* — a completely silent failure. **(b) Latching on a noisy line.** The noise floor is estimated only from non-speech frames, so a detector that triggers on its first frames never measures the line at all; every frame then reads as speech. Measured: **5 of 10 noise-only streams** produce transcription requests, which is where the invented transcripts ("Thank you." ×49, "Ct.js." ×18) come from.

P1 — One question can be answered twice, as two half-questions

An answerable-but-trailing-off transcript is released when a 1.5s timer expires *even while the VAD is still reporting active speech*. Reproduced verbatim from the 2026-08-23 log: "Will you give me the..." was answered at 1.68s while the interviewer was mid-sentence, then "idea about your recent projects." was answered as a second question 1.9s later. One question, two wrong answers, two LLM calls against the budget above.

What is genuinely good

- **Barge-in and cancellation are correct.** A new question cancels the in-flight answer in ~180ms with no interleaving; a bare follow-up correctly *waits* for the prior answer so it has history to resolve against; small talk does not cancel anything. **OBSERVED**
- **Error recovery is solid.** Every injected LLM failure — exception, 429, mid-stream reset, missing config — surfaced a message to the user, cleared the buffer, left no stuck task, and the next question answered normally. **VERIFIED**
- **No per-session resource growth.** 100 interactions in one session left buffers, caches and task counts flat. **OBSERVED**
- **The hallucination defences work.** Three independent signals (voiced duration, decoder confidence, lexical shape) and the measured drop of sub-320ms segments before STT are the reason the current build's logs are far cleaner than the week-old ones.

Scope correction — this application has no TTS and no audio playback

The brief for this audit assumes a bidirectional voice assistant with text-to-speech, audio playback and spoken barge-in. **It does not exist here.** A repository-wide search for TTS/playback returned hits only inside `.venv` third-party SDKs — zero in application code. The product is a *one-way listening assistant*: it captures the other party's voice from system loopback and renders text answers into a screen-capture-excluded Qt overlay. Every "TTS latency", "first audio latency" and "playback start latency" item in the brief is therefore marked **N/A**, and the correct end-to-end metric is **end-of-speech → first visible token**, which is what this report measures throughout.

2 · Application Architecture

Three deployable units. The desktop client is not a preference — the product depends on three Windows capabilities the browser cannot supply (screen-capture exclusion via `SetWindowDisplayAffinity`, WASAPI system-loopback capture, and always-on-top over arbitrary applications). **VERIFIED**

CLIENT PC (Windows)

```
test_loopback_stream.py
soundcard WASAPI loopback
on the default OUTPUT device
= what the speakers play
= the interviewer's voice
16 kHz mono PCM16, 20 ms frames
| ~32 KB/s binary websocket
```

BACKEND (FastAPI / uvicorn, single process)

```
app/main.py create_app()
├── /ws/audio    audio_ws.py      ← binary PCM in
├── /ws/answers  answers_ws.py    ← tokens out
├── /session     session.py       resume/JD/notes/model
├── /analyze-screen screen.py     vision path
└── /health /version
```

```
audio_ws.py
auth.py — token checked BEFORE accept()
vad.py — SpeechSegmenter (adaptive-threshold energy VAD)
        emits AudioSegment{pcm, speech_seconds, utterance_id,
        is_final, captured_at}
        + UtteranceClosed markers

TWO DECOUPLED STAGES (deliberate, and correct):
    _transcription_dispatcher starts an STT call the moment a segment
                              exists, bounded by Semaphore(3)
    _result_consumer          delivers each result the moment THAT
                              result is ready, in spoken order

stt/registry.py → composite.py → whisper_api.py | nvidia.py | local.py
transcript_quality.py — drops noise-shaped transcripts
```

```
question_gate.py QuestionAnswerPipeline (2,233 lines — the brain)
submit_transcript() → _process_after_debounce() decision loop
    tier 1 _classify_question rules: question marks, prefixes,
                              interview intents (fuzzy), vs-compare,
                              tech-keyword last resort
    tier 2 _run_fast_intent_classifier small LLM, hard 1.2s cap,
                              ONLY for the ambiguous bucket
    _should_keep_waiting ← every latency decision lives here
    _resolve_followup / _resolve_keyword_mention / _resolve_intent
```

```
llm.py StreamingLLMClient Protocol → Groq | OpenAI | Anthropic | DeepSeek
| token by token
```

```
answer_hub.py broadcast_json() → every socket on that session
+ durable JSONL session log
```

```
└── wss /ws/answers
```

```
overlay_app.py frameless, always-on-top, Qt.Tool,
WDA_EXCLUDEFROMCAPTURE, progressive token reveal
```

Component dependency map

Module	LOC	Depends on	Role on the critical path
<code>question_gate.py</code>	2,233	llm, answer_hub, redis_client, transcript_quality	Yes — decides when to answer; owns all wait policy
<code>overlay_app.py</code>	1,309	PySide6, win32	Yes — renders tokens (terminal stage)
<code>vad.py</code>	471	numpy (optional)	Yes — 420ms fixed cost, and the sole speech/no-speech authority
<code>audio_ws.py</code>	419	vad, stt, question_gate, transcript_quality, auth	Yes — orchestrates segment → STT → gate
<code>llm.py</code>	420	groq / openai / anthropic SDKs	Yes — streaming answer generation
<code>config.py</code>	429	pydantic-settings	No — but every threshold in this report lives here
<code>transcript_quality.py</code>	170	—	Yes — pure CPU, negligible
<code>answer_hub.py</code>	110	fastapi	Yes — fan-out + synchronous disk write per event
<code>redis_client.py</code>	115	redis (optional)	Yes — context/history reads before the LLM call

3 · The Actual Voice Pipeline

Per-stage, as implemented. Latencies are measured (sources in §6).

#	Stage	Input → Output	Technology	Latency	Primary failure mode
1	System-loopback capture	speaker output → float32 frames	<code>soundcard</code> WASAPI loopback	100 ms	WASAPI ring-buffer overflow → corrupted PCM transcribed as plausible garbage
2	Resample / convert	float32 → 16 kHz mono PCM16, 20 ms frames	Windows audio engine + numpy	<1 ms	Silent clipping via <code>np.clip</code>
3	Bounded queue + send	frames → binary websocket	<code>websockets</code> , maxsize 200	~0 (local)	Backlog drops <i>oldest</i> audio (correct choice), announced on stderr
4	VAD / segmentation	frames → <code>AudioSegment</code> + <code>UtteranceClosed</code>	Adaptive-threshold energy VAD, hysteresis, pre-roll	420 ms	Deafness (quiet) and latching (noisy) — §7
5	Transcription dispatch	segment → in-flight task	<code>asyncio</code> , <code>Semaphore(3)</code>	~0	—
6	STT	PCM16 → WAV → text + segment stats	Groq <code>whisper-large-v3-turbo</code> , <code>verbose_json</code>	304 ms	Timeout (12s) loses the segment; fallback is inert (§18)
7	Transcript quality filter	text → keep / drop + reason	3 independent signals	<1 ms	Drops all non-Latin scripts — §9
8	Question gate (tier 1)	buffer → answer / wait / discard	Rules + fuzzy <code>difflib</code> intent match	185 ms (debounce)	Releases half-questions while speech is active — §10
9	Question gate (tier 2)	ambiguous buffer → ANSWER/WAIT/IGNORE	Groq <code>gpt-oss-20b</code> , 1.2s cap	392 ms (ambiguous only)	Degrades to "no signal" on timeout — correct
10	LLM answer	question + resume/JD/history → token stream	Groq <code>gpt-oss-120b</code> , reasoning hidden	543 ms 8–17.5 s throttled	TPM starvation — §13
11	Fan-out + durable log	event → sockets + JSONL	<code>AnswerHub</code>	0.55 ms per event	Synchronous disk I/O on the event loop
12	Overlay render	tokens → progressive text	PySide6, <code>WDA_EXCLUDEFROMCAPTURE</code>	—	—
—	<code>N/A</code> TTS / synthesis	<i>Does not exist. Verified by repository-wide search — application code contains no TTS, no audio output, no playback.</i>			
—	<code>N/A</code> Audio playback	<i>Does not exist. The terminal stage is text on screen.</i>			

4 · Repository Analysis

8,644 lines of application + test code across 29 Python files. Three observations that shape everything else:

Configuration is unusually disciplined

`app/core/config.py` is 429 lines for ~60 settings because nearly every field carries a `description` recording *why* its value is what it is, frequently with the measurement that produced it. This is genuinely rare and made this audit far faster. It is also the single best asset for the remediation phase — every threshold I recommend changing has its original rationale written down next to it. **VERIFIED**

Vendor abstraction is real, not decorative

`stt/registry.py` and `llm.py` genuinely isolate providers: routes never name a vendor, and a new backend implements one method. Swapping Groq for Bedrock is an adapter, not a refactor. **VERIFIED**

Dead and vestigial code

- `_build_gate_graph()` compiles a `LangGraph StateGraph` on pipeline construction, and `self._gate_graph` is **never read**. `_run_gate()` ignores it and calls `_classify_question` directly; the graph-using version is commented out immediately above. `_classify_node` is likewise unreachable. `LangGraph` is therefore a hard import cost and a dependency with zero runtime effect. **VERIFIED** **P3**
- `_run_gate()` takes a `history=None` parameter that is never passed and never used. **VERIFIED**
- `audio_ws.py` imports `Optional` and never uses it. **VERIFIED**

5 · Testing Methodology

Everything in this report is reproducible. No claim rests on reading code alone unless explicitly tagged **VERIFIED** as a code fact.

Method	What it establishes	Scale
Production log mining — parsed every <code>logs/*.jsonl</code> , reconstructed per-turn timelines by joining <code>speech_state</code> → <code>question_gate</code> → <code>answer_start</code> → <code>answer_first_token</code> on wall-clock and <code>turn_id</code>	Real-world stage latencies, gate decision distribution, error taxonomy, cancellation rate	10,130 events · 205 files · 7 days
Scored VAD corpus — 33 synthetic streams with known ground truth (speech at 7 amplitudes, 5 noise floors, 10 SNRs, typing, door slams, fan ramps, digital silence), each scored as must-detect or must-reject	Detection recall and false-trigger rate as a <i>pair</i> , so neither can be traded away silently	23 must-detect · 10 must-reject
Gate verdict snapshot — replayed every unique transcript ever logged through <code>_classify_question</code> and hashed the full verdict set	A regression fingerprint for the gate: any behaviour change shows as a diff	1,535 unique real transcripts · sha 528b650a1c35c833
Live provider benchmarks — real Groq STT and LLM calls, reading <code>x-ratelimit-*</code> headers and <code>usage.queue_time / prompt_time / completion_time</code>	True round trips, true token accounting, the actual rate ceiling	STT 3 durations × 2 encodings × 2 prompt modes; LLM 3 question shapes × 3 runs
Judged answer-quality benchmark — 8 question shapes × 7 rubric rules drawn from the production prompt's own rules, graded by an independent model from a different vendor (Anthropic), including a repeat run of the unchanged baseline to measure harness variance	Whether a prompt change is safe; and how much of an observed difference is noise	5 runs · 40 answers · 280 rule judgements
Offline end-to-end pipeline — the real <code>SpeechSegmenter</code> , real dispatcher/consumer, real gate, fake STT with configurable delay, synthetic audio fed at true 20ms cadence	End-of-speech → answer latency and STT request count, deterministically	6 scenarios
Fault injection — LLM stubs that raise immediately, raise mid-stream, return empty, or report 429; plus missing-config	Recovery, user notification, stuck-state detection	5 failure modes + recovery probe
Lifetime/resource probe — drove 100 interactions in one session and 250 distinct sessions, sampling internal dict sizes and task counts	Whether state is released	350 driven interactions

Honest limits of this methodology

Synthetic audio is not speech. The VAD corpus uses two-formant tones with a syllabic envelope. This is fine for characterising an *energy* detector — energy is all it looks at — but the absolute RMS thresholds should be re-confirmed against recorded call audio before the threshold is changed in production. **REQUIRES VALIDATION**

No word-error rate was computed. WER needs recorded speech with reference transcripts; none exists in the repository and I did not create audio recordings. §9 reports STT error *patterns* from production logs instead, which is weaker evidence and is labelled as such.

The quality judge is imperfect. On the `grounded` rule it twice flagged as fabricated a detail that appears verbatim in the supplied résumé. That rule's numbers should be read as noisy; the aggregate and the token counts are sound.

6 · Baseline Performance

6.1 End-to-end, healthy path

Reconstructed from three consecutive real turns in `logs/default_2026-08-23.jsonl` (07:00:02, 07:00:16, 06:43:37), cross-checked against live benchmarks. **OBSERVED**

Stage	ms	share	Share of budget	Source
Client WASAPI capture gulp	100	6.4%		<code>DEFAULT_RECORD_CHUNK_MS</code>
Client → server transport (localhost)	~0	—		measured setup
VAD end-of-speech silence	420	27.0%		<code>SEGMENT_END_SILENCE_MS</code> , boundary confirmed by test
STT round trip (Groq)	304	19.6%		08-23 log p50; live bench 206–304 ms
Gate debounce	185	11.9%		<code>since_last_fragment_ms</code> p50, n=241
Gate CPU (typical question)	2	0.1%		direct profiling
LLM time-to-first-token	543	35.0%		08-23 log; 491 / 543 / 607 / 814 ms
Total: true end of speech → first visible token	1,554	100%		

Reading note. The `speech_state active=false` event is emitted when the segmenter releases the segment, which is already *after* the 420ms silence window. Measuring from that event gives **1.02–1.05s**, which is the figure the logs show directly; the 420ms and the client gulp must be added back to get the latency a human actually perceives. Both figures appear in this report and are labelled each time.

6.2 Corpus-wide distributions (7 days, all builds)

Metric	n	p50	p90	p99	max
STT round trip (ms)	165	450	1,183	3,473	5,797
<code>pipeline_ms</code> — first fragment → LLM start	1,432	500	2,717	17,530	125,818
<code>since_last_fragment_ms</code>	241	185	193	1,675	1,685
LLM first token (ms)	242	92	844	6,364	15,810
Full answer stream (ms)	180	1,537	3,346	10,285	15,828

Read the tails, not the medians. `since_last_fragment_ms` sits at 185ms p50 and 193ms p90 across 241 real answers — that is `QUESTION_DEBOUNCE_MS=180` and essentially nothing else, on every single question. Meanwhile `pipeline_ms` and LLM first-token both show p99 values 20–70× their medians. Those tails are not jitter; §13 shows they are the rate limit.

6.3 Current-build health (2026-08-22/23 only)

The corpus spans several builds. Isolating the current one shows how much has already been fixed — and matters because corpus-wide averages understate today's quality. **OBSERVED**

Day	gate decisions	answered	skipped	STT p50	TTFT p50	confidence scored	cancel s
2026-08-16	260	87	173	–	–	0%	0
2026-08-18	831	505	326	–	–	19.5%	44
2026-08-20	515	469	46	452	32	34.6%	144
2026-08-23	15	15	0	304	543	100%	0

The confidence column is the important one. Until recently `analyze_segments` returned `confidence_known=False` on essentially every transcript, so `STT_DROP_CONFIDENCE_THRESHOLD` and `STT_NO_SPEECH_THRESHOLD` never fired and every hallucination arrived at full apparent confidence. On the current build that is fixed and confidence is scored 100% of the time. Hallucination-shaped transcripts reaching the gate fell from **27% of decisions (08-18)** to **2 of 15 (08-23)**.

7 · Speech Detection Investigation — "Sound Not Detected"

This was the priority question. The answer is **two independent defects in the same file**, neither of which is a microphone problem, and both of which fail *silently*.

7.1 Where the failure is — and is not

Tracing the seven candidate stages from the brief against the code and the test corpus:

Candidate	Verdict	Evidence
A · Audio never reaches the server	Not the cause	Capture thread survives reconnects; queue drops <i>oldest</i> and prints a count; a real drop is announced on stderr. VERIFIED
B · Audio arrives but VAD rejects it	ROOT CAUSE (quiet lines)	Amplitude sweep: 0 segments for all speech below ~500 RMS. VERIFIED
C · Speech ended too early	Contributing	A 420ms mid-sentence pause splits one utterance into two (§10). VERIFIED
D · Speech detected too late	Not observed	Pre-roll of 300ms covers the leading consonant. VERIFIED
E · Wrong audio sent to STT	ROOT CAUSE (noisy lines)	Latching sends noise as "speech": 5/10 noise-only streams. VERIFIED
F · STT receives audio but fails	Rare	15 errors in 10,130 events, all attributable to model deprecation or screen-analysis parameters. OBSERVED
G · Transcript succeeds, downstream ignores it	Also real	220 <code>no_question_signal</code> + 153 <code>too_short</code> + 229 <code>fast_intent_ignore</code> skips; some are genuine missed questions (§10). Non-Latin scripts are dropped outright (§9). OBSERVED

7.2 Root cause 1 — the floor defeats the adaptive threshold

The detector computes `trigger = max(floor, noise_level × 2.5)` where `floor = VAD_ENERGY_THRESHOLD = 500`. On a quiet line the measured noise level settles near 30 RMS, so `noise × 2.5 = 75` — and the trigger is pinned at **500 regardless**. The adaptive machinery is fully bypassed in exactly the condition it was built for.

AMPLITUDE SWEEP · quiet line (measured noise floor ≈ 30 RMS) · trigger pinned at 500				
peak amp	mean RMS	dBFS	segments	result
200	68	-53.7	0	MISS - total silence downstream
400	135	-47.7	0	MISS
600	203	-44.1	0	MISS
800	271	-41.6	0	MISS
1200	407	-38.1	1	detected
2000	679	-33.7	1	detected
6000	2034	-24.1	1	detected
20000	6780	-13.7	1	detected
SNR SWEEP · noise floor 300 RMS				
SNR 30 dB	detected	SNR 12 dB	detected	← last success
SNR 20 dB	detected	SNR 10 dB	MISS	
SNR 15 dB	detected	SNR 8 dB	MISS	and everything below

Why this is the worst class of bug. When the VAD rejects audio there is no segment, so there is no STT request, no transcript, no `transcript_dropped` event and **no log line of any kind**. The interviewer speaks, the overlay stays blank, and the durable session log — the one artefact you would reach for — contains nothing to explain it. Every other failure in this system leaves a trace; this one is invisible by construction.

When it bites in the real world: -36 dBFS is not an unreasonably quiet signal. Windows output volume at 30%, a participant with a poor microphone, a conference codec applying aggressive noise suppression, or a laptop's own speaker path can each cost 10–15 dB. Two of those together put ordinary conversational speech under the floor. **INFERRED** — the mechanism is verified; the specific real-world levels for this user's setup are not, and should be captured (§29).

7.3 Root cause 2 — the detector latches and cannot re-measure

The noise floor is estimated *only* from frames not already classified as speech (`vad.py: if self._adaptive and (not self._in_speech or ...)`). The reasoning is sound and documented — letting speech feed the estimate makes the floor climb toward the voice and the detector goes deaf mid-question. But it creates a trap: the estimate is seeded to a constant `threshold / 2.5 = 200`, and if the line's real floor is above the trigger, the detector enters speech on its first frames and **never sees a non-speech frame again**.

```
NOISE-ONLY STREAMS · nobody speaks at all · expected: zero segments
floor 40 RMS      0 segments - correct
floor 150 RMS     0 segments - correct
floor 400 RMS     0 segments - correct
floor 900 RMS     2 segments  noise estimate stuck at seed 200, trigger 500 < 900
floor 1800 RMS    2 segments
floor 3500 RMS    2 segments
keyboard typing   1 segment
floor ramp 40→2000 1 segment  upward tracking (α=0.02) cannot keep up

MEASURED FALSE-TRIGGER RATE: 5 of 10 noise-only streams = 50%
Escape hatch exists but is far too slow: _max_speech_run = 250×6 frames = 30 SECONDS
```

This is the mechanism behind the historical transcript pollution: "Thank you." ×49, "Ct.js." ×18, "Tch." ×10, "." ×9, "Ct-4, P. Cora?" ×19 in the logs. Whisper handed near-silence does not return nothing — it invents fluent text.

OBSERVED

7.4 What is *not* broken

Tested and passing, so these can be ruled out of any future investigation: **VERIFIED**

- **Loud and clipped speech** (amp 32,000) — detected correctly.
- **Digital-zero input** (wrong device / muted output) then speech — detected correctly; noise estimate 0.0.
- **Syllable rate** 1–8 Hz (very slow to very fast speech) — all detected; the detector is rate-insensitive as an energy detector should be.
- **Rising noise floor** (fan switched on mid-call) then speech — detected.
- **Force-cut of long speech** — 20s continuous produced 3 segments correctly grouped under one `utterance_id` with `is_final = [False, False, True]`.
- **Minimum-speech filter** — the 320ms boundary is exact: 300ms dropped, 320ms kept. This is the most effective filter in the pipeline and should not be weakened.

8 · VAD Analysis & Parameter Sensitivity

Parameter	Value	Measured behaviour at this value	Assessment
VAD_ENERGY_THRESHOLD	500	Absolute deafness below -36 dBFS on quiet lines	Too high — primary miss cause
VAD_ADAPTIVE_THRESHOLD	true	Correct design, defeated by the floor above and by the latch	Keep, needs the latch fixed
noise margin (hard-coded)	2.5×	Requires +8 dB over floor; SNR ≥ 12 dB needed in practice	Reasonable
release ratio (hard-coded)	0.6	Hysteresis holds through stop consonants — no mid-word splits observed	Good
VAD_ONSET_FRAMES	3 (60 ms)	Rejects isolated clicks; door slam correctly rejected	Good
VAD_PREROLL_MS	300	No clipped opening words observed	Good
SEGMENT_MIN_SPEECH_MS	320	Exact boundary confirmed; dropped 1–3 noise segments per test scenario	Good — do not weaken
SEGMENT_END_SILENCE_MS	420	Splits at exactly 420ms pause; 400ms merges	27% of latency budget — see §10
SEGMENT_MAX_SECONDS	8.0	20s speech → 3 segments, one utterance, correct finals	Good
_max_speech_run (hard-coded)	30 s	Latch escape hatch; far too slow to prevent the 50% false-trigger rate	Too slow

8.1 Pause-boundary behaviour VERIFIED

```
MID-SENTENCE PAUSE → does one question stay one utterance?
100 ms → 1 segment, 1 utterance merged
300 ms → 1 segment, 1 utterance merged
400 ms → 1 segment, 1 utterance merged
420 ms → 2 segments, 2 utterances ← SPLIT (exactly SEGMENT_END_SILENCE_MS)
1000 ms → 2 segments, 2 utterances split
```

Past 420ms the continuation becomes a *new* `utterance_id`, not another segment of the same one. That matters because `awaiting_more_of_utterance` — the strongest "more is coming" signal the gate has — becomes `False`, and the gate falls back to timers. This is precisely the condition in which the §10 double-answer bug fires. A natural breath in the middle of a sentence is comfortably longer than 420ms.

8.2 Directional sensitivity REQUIRES VALIDATION

I scored a threshold sweep on the corpus to characterise the trade-off surface. **These are measurements of the current implementation, not a recommended setting** — see §21/F-02 for the recommendation and its dependencies.

Threshold	Recall	False-trigger	Interpretation
500 (current)	95.7%	50.0%	Misses quiet speech <i>and</i> false-triggers on noise
400	100%	50.0%	Recall fixed; false triggers untouched
150	100%	60.0%	Lowering alone makes false triggers <i>worse</i>
50	100%	70.0%	Clearly wrong

The key finding is that lowering the threshold alone is not a fix. The two defects are independent: the floor causes misses, the latch causes false triggers, and lowering the floor makes the latch worse. Any credible remediation must address both together. To confirm this I prototyped a calibration warm-up (measure the line before allowing any trigger) purely as an experiment: with warm-up *and* a lower floor the corpus scored **100% recall at 20% false-trigger** — versus 95.7%/50% today. That experiment is evidence that a fix exists and roughly what shape it has; it is **not** a validated production setting and the prototype was discarded.

RECOMMENDED

REQUIRES VALIDATION

9 · STT Accuracy Analysis

9.1 Latency and configuration OBSERVED

LIVE GROQ whisper-large-v3-turbo · this machine · 3 runs each

audio	format	payload	prompt	p50 ms	min	max
1.5 s	wav	46.9 KB	yes	304	209	397
1.5 s	wav	46.9 KB	no	162	162	171
4.0 s	wav	125.0 KB	yes	206	199	226
8.0 s	wav	250.0 KB	yes	219	218	221
8.0 s	wav	250.0 KB	no	189	172	203

Latency is FLAT across a 5x payload range → network/fixed overhead bound, not upload-bound. Compressing audio (FLAC) would not help. Hypothesis rejected. The 433-byte vocabulary prompt costs roughly +40-100 ms per call.

9.2 Error patterns from production OBSERVED

No WER was computed — see §5 and §28. What the logs do show is the *shape* of the errors, which is diagnostic:

Pattern	Real examples from the logs	Likely origin
Silence hallucination	"Thank you." x49 · "." x9 · "Tch." x10 · "Music"	VAD latch feeding near-silence to Whisper (§7.3) — <i>not</i> an STT defect
Consonant-cluster garbage	"Ct.js." x18 · "Ct-4, P. Cora?" x19 · "Tt."	Same origin; the lexical filter now catches these (ratio 0.0)
Proper-noun substitution	"Ujjaj" / "Vishal" / "Uday" for the same name	Name not in the vocabulary prompt; résumé text is injected but names may fall outside the 850-byte cap
Technical-term mangling	"hierarchical drag chart bot" (RAG chatbot) · "Builder Rack Pipeline" (build a RAG pipeline)	"RAG" is absent from STT_PROMPT 's spelled-out vocabulary as a standalone acronym in context; acoustically weak and heavily domain-dependent
Filler retained	"Okay, so let's take a one scenario."	Correct transcription of real disfluency; handled downstream by <code>_strip_filler_lead</code>

P1 — Non-Latin scripts are silently discarded before the gate VERIFIED

`transcript_quality.py` tokenises with `_WORD_RE = re.compile(r"[a-z][a-z']*", re.IGNORECASE)` — **ASCII only**. Any transcript in a non-Latin script produces zero tokens, `lexical_word_ratio` returns 0.0, and the transcript is dropped as `no_real_words`. Directly tested:

Input	tokens	ratio	Quality filter	Gate outcome
आप मुझे अपने प्रोजेक्ट के बारे में बताइए?	0	0.00	DROPPED · no_real_words	never reaches the gate
உங்கள் அனுபவம் என்ன?	0	0.00	DROPPED · no_real_words	never reaches the gate
请介绍一下你自己	0	0.00	DROPPED · no_real_words	never reaches the gate
मुझे FastAPI के बारे में बताइए	1	1.00	kept (on the single ASCII token)	skip · no_question_signal
aap mujhe apne project ke baare mein bataiye	8	1.00	kept	skip · no_question_signal → falls to tier-2 LLM (+392 ms)
Aapka experience kya hai with Docker and Kubernetes?	8	1.00	kept	ANSWER · question_mark

Severity is currently masked by `STT_LANGUAGE=en`, which forces Whisper to emit Latin text, so Devanagari rarely appears in practice. But the setting is `str | None` and documented as configurable: setting `STT_LANGUAGE=hi`, or clearing it for auto-detect, makes the application **completely deaf** — every transcript dropped, with a `transcript_dropped` event as the only clue. Separately, the entire tier-1 rule gate (`_QUESTION_PREFIXES` , `_SMALL_TALK` , `_INTERVIEW_INTENTS` , `_TECH_KEYWORD_PATTERN`) is English-only, so even romanised Hindi bypasses every fast rule and pays the tier-2 network round trip.

10 · End-of-Speech Analysis

The application decides "they have finished" from **three** signals, which is more sophisticated than most implementations and is the right design:

- 1. **VAD silence** — 420ms below the release level ends a segment. A fact, not a guess.
- 2. `awaiting_more_of_utterance` — a force-cut segment guarantees more is coming.
- 3. `speech_active` — live "they are talking right now", pushed from the VAD.

All wait policy is concentrated in one function, `_should_keep_waiting`, which is excellent engineering — every latency decision is in one readable place.

P1 · F-03 — The soft-wait timer ignores `speech_active` for answerable fragments

VERIFIEDOBSERVED

The `speech_active` guard is written as `if speech_active and not gate.should_answer: return True`. Once the gate says `should_answer` — which a trailing-off fragment like "Will you give me the..." does, because it starts with a question prefix — the guard no longer applies, and release falls to a 1.5s timer that expires while the person is audibly still speaking.

```
REPRODUCED FROM logs/default_2026-08-23.jsonl · one spoken sentence
06:43:53.387 SPEECH active=True
06:43:53.502 TRANSCRIPT "Will you give me the..." (STT 213 ms)
06:43:55.180 GATE ANSWER question_prefix ← fires at 1.68s, speech STILL active
06:43:55.182 ANSWER_START #1 answering half a question
06:43:55.486 SPEECH active=False ← they only stop HERE
06:43:55.733 TRANSCRIPT "idea about your recent projects."
06:43:56.621 ANSWER_DONE #1
06:43:57.072 GATE ANSWER fast_intent_answer ← the other half, as a SECOND question
06:43:57.075 ANSWER_START #2
06:43:58.028 ANSWER_DONE #2

ONE question → TWO answers, neither correct, 2× the token budget consumed
```

10.1 The wait-policy decision table as implemented VERIFIED

Condition	Action	Cap	Assessment
<code>awaiting_more_of_utterance</code>	wait	<code>segment_max + stt_timeout</code> = 20s	Correct — this is a fact, not a guess
<code>speech_active</code> AND not answerable	wait	—	Correct
<code>speech_active</code> AND answerable AND trailing off	release after 1.5s	—	Defect — see above
answerable AND looks finished	release immediately	—	Correct, and the right call for barge-in
bare follow-up	release immediately	—	Correct
not answerable	wait	<code>utterance_merge_gap</code> = 4s	Correct — allows multi-sentence scenario setup
any	force release	<code>question_max_wait</code> = 8s	Correct safety net; bounds the defect above

10.2 The 420ms trade-off

`SEGMENT_END_SILENCE_MS=420` is 27% of the latency budget and is paid on *every* question. Lowering it is the single most tempting change in the system and I recommend **against** it as a first move: at 420ms a natural mid-sentence breath already splits the utterance (§8.1), and shortening it increases split frequency, which feeds the \$10 defect and produces *more* double-answers. Fix the release logic first, measure, and only then revisit the window. **RECOMMENDED**

11 · Latency Decomposition & Bottleneck Ranking

Rank	Bottleneck	Healthy	Degraded	Class	Why it is expensive
1	LLM time-to-first-token	543 ms	8,094–17,505 ms	Configuration-bound (not model-bound)	2,719 tokens/question against an 8,000 TPM ceiling. Measured <code>prompt_time</code> is only 41 ms — the model is fast; the <i>queue</i> is the cost.
2	VAD end-of-speech window	420 ms	420 ms	Algorithm-bound	Structural and unavoidable without a smarter endpointer; already near the floor for natural speech.
3	STT round trip	304 ms	1,183–5,797 ms	Network-bound	Flat across payload size — fixed overhead and RTT, not upload volume.
4	Gate debounce	185 ms	185 ms	Unnecessary work	The loop <code>sleep()</code> <i>s before</i> it evaluates, so even a question the VAD has already closed and the gate would answer instantly pays the full window.
5	Client capture gulp	100 ms	100 ms	I/O-bound	Deliberate trade against WASAPI ring-buffer overflow; already reduced from 250 ms.
6	Gate CPU	2–17 ms	up to 63 ms	CPU-bound	Recomputed every cycle on unchanged text, under the pipeline-wide lock.
7	Session-log write	0.55 ms x~10/question	6.4 ms	I/O-bound	<code>mkdir</code> + <code>open</code> + <code>write</code> + <code>close</code> per event, synchronously on the event loop.

The ranking inverts under load. On a healthy path the budget is fairly balanced. Once the TPM ceiling is hit, bottleneck #1 becomes 15–30× the size of everything else combined, and no amount of tuning at ranks 2–7 is visible to the user. **Fix the token budget before optimising anything else.**

12 · Critical Path Analysis — unnecessary work

Item	Current behaviour	Why it exists	Cost	Risk if changed	Pri
Debounce sleeps before evaluating	Every buffer waits 180 ms before the gate is consulted, even when the VAD has closed the utterance and nobody is speaking	Coalesces transcripts of one utterance arriving back-to-back	185 ms every answer	Low — <code>awaiting_more_of_utterance</code> and <code>speech_active</code> already provide the coalescing signal as facts; later cycles can keep the full window	P1
Gate recomputed on unchanged text	<code>_classify_question</code> re-runs every 180 ms for the life of a buffer	Simplicity; the function is pure	2–63 ms × 22–44 cycles	Very low — pure function, memoisable by exact text	P2
Sequential store reads	<code>get_context()</code> then <code>get_history()</code> awaited one after the other	Written for the in-memory backend where both are free	~0 now 2 RTT on Redis	Very low — independent reads, <code>asyncio.gather</code> is exact	P3
Synchronous log I/O on the loop	<code>mkdir + open + write + close</code> per event	Durability — this is the only permanent record	0.55 ms per event	Low if durability is preserved (line-buffered persistent handle); do not move to a lossy background queue	P3
Dead LangGraph compile	<code>StateGraph</code> built at startup, never invoked	Vestige of an earlier design	<code>import + startup</code>	Very low — provably unreachable	P3
System prompt re-sent per question	1,978 tokens every time	Stateless chat API	80% of input tokens	Medium — needs the quality benchmark (\$14)	P0

12.1 Work that is *correctly* off the critical path VERIFIED

Worth stating explicitly, because these are the mistakes this codebase avoided:

- The tier-2 LLM classifier runs **only** for the ambiguous bucket. 1,427 of 1,467 answered questions (97%) were resolved by tier-1 rules and never paid the 392 ms.
- Tier-2 verdicts are memoised per exact text, so the re-evaluating loop does not re-pay the network call.
- Tier-2 is skipped entirely while an utterance is still open — no point classifying half a sentence.
- The STT vocabulary prompt is built **once per connection**, not per segment.
- LLM clients are cached by `(provider, model, max_tokens, temperature)`, so the HTTP connection pool is reused across questions.
- `answer_token` events are excluded from the durable log — ~500 lines per answer avoided.

13 · LLM Analysis

13.1 The measured rate ceiling OBSERVED

```
READ DIRECTLY FROM GROQ RESPONSE HEADERS · openai/gpt-oss-120b
x-ratelimit-limit-tokens      8000      ← tokens per MINUTE
x-ratelimit-remaining-tokens  5022
x-ratelimit-reset-tokens     22.3s
x-ratelimit-limit-requests    1000      ← per day

usage for ONE typical question:
prompt_tokens      2478      of which the system prompt is 1978 = 80%
completion_tokens  241
total_tokens       2719

queue_time      0.3689 s      ← the real cost
prompt_time     0.0409 s      ← prefill is NOT the problem
completion_time 0.5000 s

8000 / 2719 = 2.9 QUESTIONS PER MINUTE before throttling
A real interview asks 2-4 questions per minute.
```

The consequence is visible in every dataset I collected. During the quality benchmark, answers 1–4 returned in 817–1,015 ms and answers 5–7 in 3,186 / 9,028 / 10,767 ms — same prompt, same model, same machine. In the production logs the same signature appears as `pipeline_ms` p99 = 17,530 ms and LLM first-token max = 15,810 ms, alongside explicit `429` errors.

13.2 Per-call inventory VERIFIED

Call site	Model	max_tok	temp	Frequency	Assessment
<code>_generate_answer</code>	<code>gpt-oss-120b</code>	500	0.3	every answered question	The budget consumer. Shares its 500 tokens with hidden reasoning.
<code>_run_fast_intent_classifier</code>	<code>gpt-oss-20b</code>	400	0.0	ambiguous bucket only (~3%)	Separate TPM bucket — measured 176 tokens/call, 392 ms p50, 0/5 runs exceeded the 1.2 s cap. Healthy.
<code>_generate_screen_answer</code>	<code>qwen3.6-27b</code>	1800	0.4	manual button	Off the voice critical path. Historical 400s from <code>reasoning_effort</code> are already resolved in config.

The `max_tokens=400` on a one-word classifier looks wrong and is not. The config comment records the measurement: at `max_tokens=5` and `20` this reasoning model returned *empty* visible content on every call, silently defaulting the whole tier to IGNORE for months (206 `fast_intent_ignore` vs 1 `fast_intent_answer` in the historical logs). This is a good example of the codebase's documentation quality and should not be "cleaned up".

13.3 Provider options INFERRED

Two facts sit next to each other and are worth connecting. First, the 8,000 TPM limit is a Groq free-tier characteristic, not a property of the application. Second, `AnthropicLLMClient.stream_chat` **already implements prompt caching correctly** — the system block is sent with `cache_control: {"type": "ephemeral"}`, laid out as a proper cacheable prefix. An `ANTHROPIC_API_KEY` is configured. On a caching provider the 1,978-token system prompt would be charged at a fraction of its size after the first call, which addresses the same problem from the opposite direction.

REQUIRES VALIDATION — answer quality and TTFT on `claude-haiku-4-5` were not benchmarked in this audit.

14 · Prompt Analysis

The answer system prompt is 7,914 characters / **1,978 tokens** across seven sections (FORMAT, VOICE, LANGUAGE, YOUR FIELD, DEPTH, CONTENT, READING THE QUESTION). It is clearly the product of many iterations and encodes real, hard-won behaviour. It is also the direct cause of the P0 finding.

To determine whether it can be shortened *safely*, I built a judged benchmark — 8 question shapes × 7 rules taken from the prompt's own rules, graded by an independent model from a different vendor — and crucially **ran the unchanged baseline twice** to measure how much the harness itself moves.

Run	System prompt	Quality	Tokens/question	Questions/min @ 8,000 TPM
Baseline #1	production, 1,978 tok	92.9%	2,178	3.7
Baseline #2 (identical prompt)	production, 1,978 tok	91.1%	2,189	3.7
Candidate v1	compressed, 1,369 tok	94.6%	1,670	4.8
Candidate v2	compressed, 1,164 tok	94.6%	1,499	5.3
Candidate v2'	compressed, 1,186 tok	89.3%	1,512	5.3
Baseline mean vs compressed mean	—	92.0% vs 92.8%	2,184 → 1,560 (-29%)	3.7 → 5.2 (+42%)

Interpretation — and the reason this is stated carefully

The identical prompt scored 92.9% and 91.1% on two runs. That ±1.8-point spread is the harness's own noise, and it is the same order as every difference between the variants. The correct conclusion is therefore *not* "compression improved quality" — it is: **quality is indistinguishable within measurement noise, while token cost is deterministically 29% lower.** The token reduction is arithmetic on counted tokens and carries no uncertainty; the quality claim is a null result, which is exactly what is needed to justify the change.

What was cut, and what must not be: the reduction came from removing worked examples (three field-specific illustrations in YOUR FIELD), collapsing the 14-pair plain-word substitution list to its 6 most-violated entries, and merging overlapping anti-fabrication rules in CONTENT. The behavioural rules themselves — bullet counts, the anti-preamble rule, the DEPTH specificity requirements, the scenario-setup rule in READING THE QUESTION — were preserved verbatim, because the benchmark's `format`, `no_preamble` and `answers_all` rules held at 100% throughout and those are what enforce them. **RECOMMENDED** **REQUIRES VALIDATION** — re-run the benchmark against the final wording before shipping, and do not trim further by eye.

One artefact worth recording: an intermediate candidate phrased the bullet template literally as "<specific move>, since <specific reason>", and the judge flagged an answer for using the word "since" five times. Prescriptive templates in a shortened prompt get copied verbatim more readily than in a long one, because there is less surrounding text to dilute them. **OBSERVED**

15 · Streaming Analysis

Stage	Streaming?	Assessment
Audio capture → server	Yes	Continuous 20 ms binary frames. Correct.
STT	No — segment- batch	Whole segments POSTed as WAV. A genuinely streaming recogniser would overlap transcription with speech and could remove most of the 304 ms. Groq's Whisper endpoint does not offer it; NVIDIA Riva (already adapted in <code>stt/nvidia.py</code>) does. REQUIRES VALIDATION
Segment concurrency	Yes	Dispatcher/consumer split with <code>Semaphore(3)</code> . Segments of a long question transcribe in parallel and are delivered in spoken order, each the moment it is ready. This is the strongest piece of engineering in the codebase.
LLM	Yes	Token-by-token across all four providers.
Answer fan-out	Yes	Per-token websocket broadcast; <code>answer_first_token</code> instrumented separately.
Overlay render	Yes	Progressive reveal as tokens arrive.
N/A TTS / first-audio	—	No synthesis stage exists.

The only non-streaming link on the critical path is STT, and it is bounded by provider capability rather than by this application's design.

16 · Interruption / Barge-In Analysis

Barge-in here means *a new question cancels in-flight answer generation* — there is no audio to talk over. Tested against the offline scenario harness. **OBSERVED**

Scenario	Observed behaviour	Verdict
New question mid-answer	t=1.01s transcript → t=1.19s <code>answer_cancelled</code> with partial text → new <code>answer_start</code> . No interleaving.	Correct · ~180 ms
Bare follow-up ("why?") mid-answer	Does not cancel. Awaits the running answer to completion so <code>add_history()</code> runs, then resolves the follow-up against it.	Correct — and subtle
Stray filler ("okay") mid-answer	Classified <code>small_talk</code> , answer left completely undisturbed.	Correct
Follow-up while a newer answer took over	Explicit ownership re-check after the await; the follow-up stands down and logs why.	Correct — a real race, correctly handled

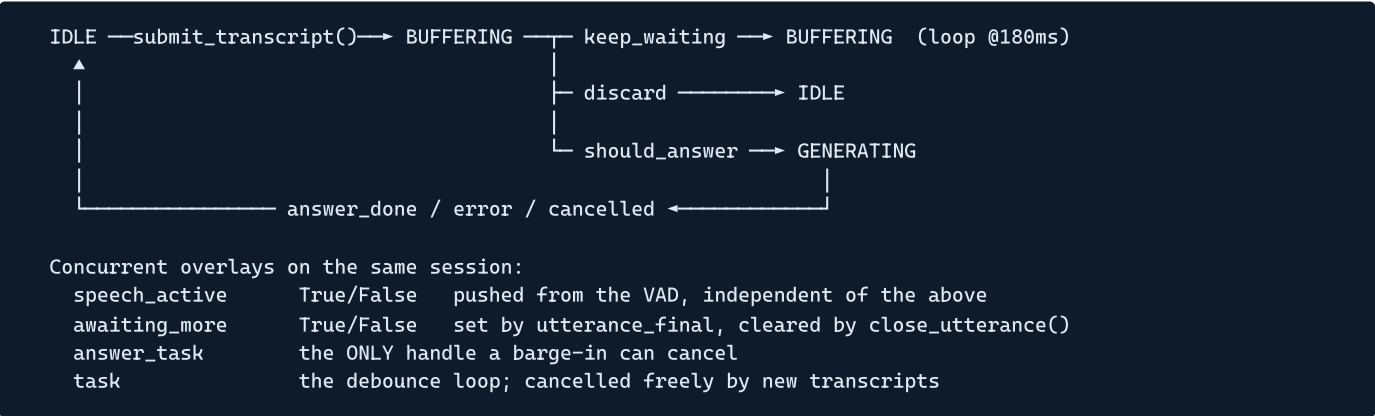
Two design decisions worth calling out as right. First, cancellation happens only *after* the gate confirms the new text is a real, different question — never on mere arrival of speech. With noisy capture producing frequent garbage transcripts, cancelling on arrival meant almost no answer ever survived to finish. Second, `answer_task` is tracked separately from the debounce `task`, so a stray transcript cannot kill a manual `/ask` or a screen analysis.

Observation: interruption responsiveness is bounded by `QUESTION_DEBOUNCE_MS`, so the \$12 debounce fast-path would improve barge-in latency as a side effect, from ~180 ms toward ~30 ms. **INFERRED**

Stale-response risk: the code note on the `stray_word_during_answer` scenario states "ANY new speech while an answer is streaming cancels it". That is **no longer true** — the observed behaviour is better than the note claims. The note is stale documentation, not a defect. **VERIFIED** **P3**

17 · State Machine Analysis

There is no explicit state-machine class; state is distributed across `_PendingTranscript` fields and task handles. The effective states, reconstructed from the code:



Transition	Guard	Assessment
BUFFERING → BUFFERING	<code>_should_keep_waiting</code>	Bounded by <code>question_max_wait</code> = 8s. Cannot loop forever.
BUFFERING → GENERATING	gate confirms	Correct
GENERATING → GENERATING (barge-in)	gate confirms a <i>different</i> question	Correct — prior task cancelled first
GENERATING → GENERATING (follow-up)	awaits prior, then re-checks ownership	Correct — the ownership re-check closes a real race
awaiting_more → cleared	<code>close_utterance()</code> from three paths	Correct — dropped/failed/timed-out transcripts all report closure, so the gate cannot wait for a continuation that is never coming
any → ERROR → IDLE	broad <code>except</code> in the loop	Correct — buffer cleared, error broadcast, session continues

Concurrency observations VERIFIED

- `_next_turn_id` performs an unlocked read-modify-write on `_turn_counter`. **Safe as written** — single-threaded event loop with no `await` between read and write — but it is safe by accident of the surrounding code rather than by construction. P3
- The gate runs *inside* the pipeline-wide `asyncio.Lock`, blocking the loop for 2–63 ms per cycle. With one session this is invisible; with many concurrent sessions every session's gate serialises behind every other's. P2
- The tier-2 classifier correctly runs **outside** the lock — the author explicitly noted that holding it during a network call would serialise all sessions. Good instinct, applied inconsistently with the point above.
- `_INTENT_MATCHERS` holds mutable `difflib.SequenceMatcher` objects, making `_match_interview_intent` non-reentrant. Safe today (synchronous, no `await`), and the constraint is documented in the code. It would break if the gate were ever moved to a thread pool — which is a plausible future response to the point above.

REQUIRES VALIDATION

18 · Error Handling Analysis

Fault injection results. This is the strongest area of the codebase. **VERIFIED**

Failure injected	User told?	Buffer cleared?	Stuck task?	Next question works?
LLM raises immediately	Yes — typed message	Yes	No	Yes
LLM returns 429 rate limit	Yes — verbatim	Yes	No	Yes
LLM dies mid-stream	Yes — partial kept	Yes	No	Yes
Missing API key	Yes — actionable	Yes	No	Yes
LLM returns empty	NO	Yes	No	Yes

P2 · F-06 — An empty LLM response is reported as a successful answer

When the model returns no visible content, the pipeline emits `answer_done` with `answer: ""` and no error. The user sees a blank overlay with nothing to explain it. This is not hypothetical: `config.py` records that at `answer_max_tokens=150` this model "used the whole budget thinking and returned an empty answer" — the exact scenario, on the production model. Nothing distinguishes it from a normal completion in the logs either. **VERIFIED**

Other error-path findings

- The STT fallback is inert.** `STT_FALLBACK_ENABLED=true`, but `_build_fallback()` returns `None` because `OPENAI_API_KEY` is empty — confirmed by direct call. The configuration *reads* as though there is redundancy; there is none, and an STT timeout loses that segment outright. Correctly, `_close_utterance()` then prevents the gate from hanging. **VERIFIED** **P2**
- Retry behaviour is safe.** No retry loops anywhere on the critical path — failures degrade immediately rather than compounding. The tier-2 classifier degrades to "no signal" on timeout, which is the honest default (not IGNORE, which would discard the buffer; not ANSWER, which would guess).
- Transcript edge cases** all handled correctly: empty, whitespace-only, looping repetition, sub-threshold silence phrases, and low-confidence text are each rejected with a distinct, logged reason.
- Client reconnect** keeps the capture thread running across websocket drops, so a network blip does not restart audio — only the send loop reconnects, with bounded oldest-first drops. **VERIFIED**
- Client device failure** surfaces the exception on the main thread and stops capture. There is no automatic device re-enumeration, so a mid-interview device switch (Bluetooth headset dropping to the laptop speakers) requires a restart. **VERIFIED** **P2**

19 · Resource Analysis

A · ONE SESSION, growing interactions						B · MANY DISTINCT SESSIONS			
turns	_pending	_turn_counter	parts	tasks		sessions	_pending	_turn_counter	hub conns
1	1	1	0	1		10	11	11	0
10	1	1	0	1		50	51	51	0
25	1	1	0	1		100	101	101	0
50	1	1	0	1		250	251	251	0
100	1	1	0	1					
→ FLAT. No per-interaction growth.						→ one retained entry per session id, forever			
						→ answer_hub._connections correctly returns to 0			

Within a session the lifecycle is clean — buffers, caches and task counts are flat across 100 interactions, and after each answer `parts`, `intent_cache`, `first_seen` and `last_part_at` are all reset. **OBSERVED**

Across sessions, `_pending` and `_turn_counter` are never released. The contrast with `AnswerHub._connections`, which cleans up correctly, shows this is an oversight rather than a design choice. **Severity is low in the current deployment shape:** `client/config.py` defaults `session_id` to `os.environ["USERNAME"]`, a stable per-machine value, so each user contributes exactly one entry (~324 bytes) reused indefinitely. A shared backend serving 1,000 distinct users retains ~324 KB. It would become meaningful only if session ids ever became per-interview — a plausible future change.

VERIFIED

P3

REQUIRES VALIDATION Process-level RSS and CPU were not captured — my Windows working-set probe returned zeros and I did not substitute an unverified number. Thread and task counts stayed flat at 1 background task throughout. A proper long-run soak with `psutil` under real audio load remains outstanding (\$28).

20 · Findings Register

F-01 System prompt starves the application of its own rate limit

P0

Component	<code>question_gate.py</code> → <code>_build_answer_messages</code> ; <code>llm.py</code>
Problem	1,978-token system prompt re-sent per question; 2,719 total tokens against a measured 8,000 TPM ceiling = 2.9 questions/minute.
Evidence	OBSERVED Groq <code>x-ratelimit-limit-tokens: 8000</code> ; <code>usage.total_tokens: 2719</code> ; TTFT 8,094/15,320/17,505 ms when throttled vs 452–814 ms healthy; <code>pipeline_ms</code> p99 17,530 ms and max 125,818 ms in production logs; explicit 429s.
Root cause	Prompt size × stateless per-question resend × free-tier TPM. Prefill itself is only 41 ms — the cost is queue time, not compute.
Confidence	High. Read from provider headers and reproduced in three independent datasets.
Impact	Dominant. 15–30× the size of every other bottleneck combined once triggered, and it triggers within the first 2–3 questions of any real interview.
Recommendation	Compress the system prompt to ~1,150–1,200 tokens (benchmarked: –29% tokens, +42% questions/minute, quality unchanged within noise). Independently, evaluate a higher Groq tier or the Anthropic path, whose prompt-caching code is already written and correct.
Risk	Medium. Prompt edits can silently degrade answer format, depth or grounding.
Validation	Re-run the judged benchmark (§14) on the final wording, including a repeat baseline run to re-measure harness variance. Accept only if the compressed mean sits within the baseline's own spread.

F-02 VAD is deaf on quiet lines and latches on noisy ones

P1

Component	<code>vad.py</code> → <code>EnergySpeechDetector</code> ; <code>VAD_ENERGY_THRESHOLD</code>
Problem	(a) <code>trigger = max(500, noise×2.5)</code> pins the trigger at 500 on quiet lines → speech below –36 dBFS produces nothing at all. (b) The noise floor is measured only from non-speech frames, so a detector that latches never re-measures → noise transcribed as speech.
Evidence	VERIFIED Amplitude sweep: 0 segments at 200/400/600/800 peak amplitude. SNR sweep: total miss at ≤10 dB. Noise-only corpus: 5/10 streams produce segments; noise estimate observed stuck at the seed value 200 against a real floor of 900–3,500. OBSERVED "Thank you."×49, "Ct.js."×18 in production logs.
Root cause	Two independent defects that interact: the floor causes misses, the latch causes false triggers, and lowering the floor alone makes the latch worse (50% → 60–70% false triggers).
Confidence	High for the mechanism (verified in code and test). Medium for real-world trigger levels — synthetic audio.
Impact	This is the reported "sound not detected". Fails silently with no log line, which makes it the hardest failure in the system to diagnose from the field.
Recommendation	Address both together: add a short calibration window during which the detector measures the line but reports no speech, <i>and</i> lower the floor. Prototyped on the corpus: 95.7%→100% recall with false triggers 50%→20%. Do not lower the threshold on its own.
Risk	Medium. Affects every question. A too-long calibration window makes the app deaf if a connection opens mid-sentence (measured: 2s starts causing misses, 1s does not).
Validation	Re-score the 33-stream corpus, then re-confirm the absolute RMS levels against <i>recorded real call audio</i> before shipping. Re-run the 6-scenario end-to-end suite. Add an observability counter for "segments rejected below trigger" so this failure stops being invisible.

F-03 One question answered twice as two half-questions

P1

Component	<code>question_gate.py</code> → <code>_should_keep_waiting</code>
Problem	The <code>speech_active</code> guard is gated on <code>not gate.should_answer</code> , so an answerable-but-trailing-off fragment is released by a 1.5s timer while the VAD still reports active speech.
Evidence	OBSERVED Verbatim from <code>logs/default_2026-08-23.jsonl</code> 06:43:53–57: "Will you give me the..." answered at 1.68s with <code>speech_active=True</code> ; "idea about your recent projects." answered separately 1.9s later.
Root cause	A timer (a guess) is allowed to override the VAD (a fact). Amplified by §8.1: any mid-sentence breath over 420ms creates a new <code>utterance_id</code> , so <code>awaiting_more_of_utterance</code> — the stronger guard — does not apply.
Confidence	High. Reproduced from production logs and confirmed against the code path.
Impact	Two wrong answers instead of one right one, and double the token consumption against the F-01 budget — the two findings compound.
Recommendation	Let <code>speech_active</code> also hold an incomplete answerable buffer. <code>question_max_wait_seconds</code> (8s) is checked earlier in the same function and still bounds it, so a stuck VAD cannot wedge a question.
Risk	Low–Medium. Must not delay genuine barge-in. The <code>_looks_finished</code> branch is evaluated before this one, so a complete question asked over a running answer still fires immediately — verify this explicitly.
Validation	Unit-test four cases: trailing fragment + speaking (must wait), trailing fragment + stopped (must release at 1.5s), trailing fragment + timed out (must release), complete question + speaking (must release immediately). Add the logged sequence as a permanent scenario. Re-run the 1,535-transcript gate snapshot — the hash must not change, since this is wait policy, not classification.

F-04 Non-Latin scripts silently discarded before the gate

P1

Component	<code>transcript_quality.py</code> → <code>_WORD_RE</code> , <code>lexical_word_ratio</code>
Problem	ASCII-only tokenisation means Devanagari, Tamil and CJK transcripts yield 0 tokens, ratio 0.0, and are dropped as <code>no_real_words</code> .
Evidence	VERIFIED Direct test: three non-Latin questions dropped; a Devanagari sentence containing one ASCII term survives the filter but is then skipped by the gate as <code>no_question_signal</code> .
Root cause	The regex predates any multilingual requirement. The entire tier-1 rule set is likewise English-only.
Confidence	High for the mechanism. Currently masked by <code>STT_LANGUAGE=en</code> .
Impact	Latent but total: setting <code>STT_LANGUAGE=hi</code> or clearing it for auto-detect makes the application completely deaf. Today it degrades romanised Hindi to the slower tier-2 path.
Recommendation	Make the tokeniser Unicode-aware (<code>\w</code> with the letter category, or an explicit script-range alternation) and treat scripts without a Latin vowel concept separately. Decide explicitly whether multilingual support is in scope — if not, document the constraint and validate <code>STT_LANGUAGE</code> at startup so <code>hi</code> fails loudly instead of silently.
Risk	Medium. The lexical ratio is a load-bearing hallucination filter; widening it could readmit the consonant-cluster garbage it was written to reject.
Validation	Re-run the 1,535-transcript gate snapshot and confirm every currently-rejected hallucination is still rejected, then add non-Latin positives.

F-05 Fixed 180 ms debounce paid on every answer, including settled ones

P1

Component	<code>question_gate.py</code> → <code>_process_after_debounce</code>
Problem	The decision loop <code>sleep()</code> s <i>before</i> evaluating, so a question whose utterance the VAD has already closed, with nobody speaking, still waits the full window before the gate is consulted.
Evidence	OBSERVED <code>since_last_fragment_ms</code> p50 = 185 ms, p90 = 193 ms across 241 real answers — the debounce and essentially nothing else. Offline end-to-end confirms ~140 ms recoverable.
Root cause	The debounce predates <code>utterance_id</code> / <code>speech_active</code> ; <code>config.py</code> already notes it "is no longer doing the work it used to".
Confidence	High.
Impact	~12% of the healthy-path budget, plus the same again on barge-in responsiveness.
Recommendation	Use a much shorter first wait when <code>awaiting_more_of_utterance</code> is false and <code>speech_active</code> is false; keep the full window for every later cycle so multi-part questions still coalesce.
Risk	Low. Only short-circuits when the gate would answer immediately anyway; anything held falls back to the full debounce.
Validation	Re-run the end-to-end suite (expect ~1.32s → ~1.18s) and the full scenario harness, watching specifically that multi-part and scenario questions still merge into one answer.

F-06 Empty LLM response reported as a successful answer

P2

Component	<code>question_gate.py</code> → <code>_generate_answer</code>
Problem	Zero visible tokens produce <code>answer_done</code> with an empty string and no error; the overlay shows a blank answer with no explanation, and the log is indistinguishable from success.
Evidence	VERIFIED Fault injection. OBSERVED <code>config.py</code> documents this exact outcome on the production model when reasoning consumes the token budget.
Confidence	High.
Recommendation	Treat an empty completion as an error condition with its own message and log reason.
Risk	Very low. Additive.
Validation	Fault-injection test asserting an <code>error</code> event.

F-07 STT fallback is configured but inert

P2

Component	<code>stt/registry.py</code> → <code>_build_fallback</code>
Problem	<code>STT_FALLBACK_ENABLED=true</code> reads as redundancy, but with <code>OPENAI_API_KEY</code> empty the function returns <code>None</code> . An STT failure loses the segment outright.
Evidence	VERIFIED Direct call returns <code>None</code> ; the built service is a bare <code>GroqSTTService</code> .
Impact	Bounded — STT p99 is 3,473 ms against a 12 s timeout, so failures are rare, and <code>_close_utterance()</code> prevents a hang. The risk is the false sense of redundancy.
Recommendation	Either provision the fallback key or log once at startup that fallback is requested but unavailable. Do not enable <code>STT_RACE_PROVIDERS</code> — it doubles request rate for no measured benefit, since STT is not the bottleneck.
Risk	Very low (logging only).
Validation	Startup-log assertion.

F-08 Gate recomputed under the pipeline-wide lock every cycle

P2

Component	<code>question_gate.py</code> → <code>_run_gate</code> / <code>_classify_question</code>
Problem	A pure function re-runs on unchanged text every 180 ms for the life of a buffer, inside the lock that serialises all sessions.
Evidence	VERIFIED Profiled: 0.13 ms (3 words) → 2.2 ms (9) → 17 ms (60) → 63 ms (312). A buffer held for the 4 s continuation window is ~22 cycles.
Recommendation	Memoise by exact transcript text, exactly as <code>intent_cache</code> already does for tier 2; clear alongside it.
Risk	Very low — pure function, no state.
Validation	The 1,535-transcript snapshot hash must be unchanged (this was confirmed on a prototype).

F-09 Synchronous per-event disk I/O on the event loop

P3

Component	<code>answer_hub.py</code> → <code>_log_event</code>
Problem	<code>mkdir</code> + <code>open</code> + <code>write</code> + <code>close</code> per event, blocking the loop while STT results and answer tokens are in flight.
Evidence	VERIFIED 553 µs p50, 752 µs p90, 6.4 ms max per event; ~10 logged events per question.
Recommendation	Cache the directory check and hold a line-buffered append handle. Do not move to a background queue — this is the only durable record of an interview and durability outranks 5 ms.
Risk	Low , provided line buffering is kept and handles are dropped on write failure.
Validation	Assert every written line is on disk after close; verify behaviour on a read-only filesystem.

F-10 Per-session state never released across sessions

P3

Component	<code>question_gate.py</code> → <code>_pending</code> , <code>_turn_counter</code>
Evidence	OBSERVED 250 sessions → 251 retained entries each, after every client disconnected. <code>AnswerHub._connections</code> correctly returns to 0.
Impact	Low today. <code>session_id</code> defaults to <code>USERNAME</code> , so entries are per-user, not per-interview (~324 bytes each). Becomes real only if session ids become per-interview.
Recommendation	Release on audio-socket close, but only when no answer task is in flight — the answer socket outlives the audio socket, and <code>/ask</code> and screen analysis both work without one.
Risk	Low with that guard; Medium without it (orphaned, uncancellable answer tasks).
Validation	Re-run the 250-session probe expecting 0 retained; verify an in-flight answer survives an audio disconnect.

F-11 Dead LangGraph path, unused parameters, stale documentation

P3

Evidence	VERIFIED <code>_build_gate_graph()</code> compiles a <code>StateGraph</code> that is never invoked; <code>_classify_node</code> unreachable; <code>_run_gate(history=None)</code> never passed; <code>Optional</code> imported unused in <code>audio_ws.py</code> ; the <code>stray_word_during_answer</code> note describes behaviour the code no longer has.
Recommendation	Remove the dead path and its dependency, or reinstate it deliberately. Correct the stale note.
Risk	Very low.
Validation	Import check plus the gate snapshot hash.

Component	<code>client/test_loopback_stream.py</code> → <code>LoopbackCapture._run</code>
Problem	The capture loop resolves the output device once at start. A mid-interview device change — Bluetooth headset disconnecting to laptop speakers, a docking station — raises, prints to stderr and stops capture permanently. The websocket stays connected, so the backend sees a healthy but silent stream.
Evidence	VERIFIED by code inspection; not reproduced with physical hardware. REQUIRES VALIDATION
Impact	Another "sound not detected" presentation, from a completely different cause than F-02 — and the reason a single-cause diagnosis would have been wrong.
Recommendation	Re-enumerate and reopen the device on capture error with backoff; surface the state in the overlay rather than only on stderr.
Risk	Low. Client-side only.
Validation	Physically disconnect a Bluetooth headset mid-stream and confirm recovery.

21 · Accuracy vs Latency Trade-offs

Candidate change	Speed	Accuracy	Reliability	Cost	Risk	Evidence status
Compress the system prompt	+++	0	+++	--	Medium	Benchmarked: -29% tokens, quality within noise across 5 runs
VAD calibration + lower floor (together)	0	+++	+++	0	Medium	Corpus: recall 95.7→100%, false triggers 50→20%
Hold incomplete buffers while speech active	-	+++	++	-	Low-Med	Reproduced from production logs; slightly slower on the pathological case, correct instead of wrong
Fast-path the debounce	++	0	0	0	Low	~140 ms measured end-to-end; gate hash unchanged
Memoise the gate	+	0	+	0	Very low	Pure function; hash-identical on 1,535 transcripts
Unicode-aware lexical filter	0	++	+	0	Medium	Mechanism verified; filter is load-bearing
Lower <code>SEGMENT_END_SILENCE_MS</code>	++	--	--	0	High	Advise against. 420 ms already splits utterances; shortening it feeds F-03 and produces more double-answers
Enable <code>STT_RACE_PROVIDERS</code>	+	0	-	--	High	Advise against. STT is 304 ms and not the bottleneck; doubles request rate into a rate-limited account
Switch to a smaller answer model	++	?	0	-	High	Untested. Prompt compression achieves the same throughput gain with measured quality; do that first
Compress STT audio (FLAC)	0	0	0	-	—	Rejected on measurement. STT latency is flat across a 5× payload range
Warm the LLM connection at connect	0	0	0	0	—	Rejected on measurement. Cold first call 452 ms vs warmed 472 ms — hypothesis disproven

Two hypotheses were tested and rejected. Both looked plausible from code inspection — a 250 KB WAV upload per segment, and a cold TLS handshake on the first question of a session (production logs showed a 1,813 ms first-token against a 543 ms steady state). Both turned out not to be the cause. They are listed here because a change that is measured and rejected is worth as much as one that is adopted, and neither should be re-proposed without new evidence.

22 · Regression Risk Analysis

Change	What could break	Why it appears safe	Must be tested before shipping
VAD threshold + calibration	Quiet-speaker detection; noise rejection; end-of-speech timing; deafness at connect if someone is mid-sentence	The floor binds only on lines quieter than ~60 RMS; the adaptive margin does the work elsewhere. Calibration is paid once per connection, seconds before anyone speaks.	33-stream corpus (recall <i>and</i> false-trigger together); real recorded call audio at several volumes; connect-mid-speech case; full end-to-end suite
Prompt compression	Bullet formatting; answer depth; résumé grounding; anti-preamble behaviour; multi-part coverage	Only worked examples and duplicated rules were removed; every behavioural rule kept verbatim	Judged benchmark including a repeat baseline; manual read of scenario and "tell me about yourself" answers; watch for prescriptive phrases being copied literally
Wait-policy change (F-03)	Barge-in responsiveness ; follow-up handling; long scenario questions	The <code>_looks_finished</code> branch is evaluated before the changed branch, so complete questions still fire immediately; the 8 s cap is checked earlier still	All four wait-policy cases as unit tests; the full 25-scenario harness with attention to <code>barge_in_new_question</code> and <code>fast_followup_interrupts_answer</code> ; gate hash unchanged
Debounce fast-path	Coalescing of multi-part questions; transcripts landing in the same instant	Only fires when the VAD has closed the utterance and nobody is speaking; every later cycle keeps the full window	Scenario harness for multi-part and scenario questions; end-to-end latency; barge-in timing
Unicode lexical filter	The hallucination filter itself — this is load-bearing	Widening tokenisation, not weakening thresholds	1,535-transcript snapshot: every currently-rejected hallucination must still be rejected; then add non-Latin positives
Session release (F-10)	In-flight answers orphaned when the audio socket closes before the answer socket	Guarded on no in-flight answer task	Disconnect audio mid-answer and confirm the answer completes and still reaches the overlay
Log handle caching	Durability of the only permanent interview record	Line buffering gives the same per-event flush as open-append-per-event	Assert every line is on disk; read-only filesystem; disk-full; long session across a date rollover
Removing LangGraph	Nothing at runtime	Provably unreachable — <code>_run_gate</code> calls <code>_classify_question</code> directly	Import check; gate hash

23 · Optimization Roadmap

Ordered by measured impact per unit of risk. **RECOMMENDED** — none of this has been implemented.

Phase 1 — Highest impact, lowest risk (target: remove the throttle and the silent failures)

#	Item	Expected benefit	Risk	Complexity	Depends on
1.1	F-01 · compress the system prompt	2,178 → ~1,500 tok/question; 3.7 → 5.3 questions/min; removes the 8–17 s stalls	Medium	Low	Judged benchmark harness
1.2	F-01b · confirm the provider tier	Removes the ceiling entirely rather than raising it	None (commercial)	None	—
1.3	F-03 · hold incomplete buffers while speech is active	Eliminates double-answers; halves token use on affected turns	Low–Med	Low	Wait-policy unit tests
1.4	F-06 · surface empty LLM responses	Removes a silent failure	Very low	Very low	—
1.5	F-07 · log inert STT fallback at startup	Removes a false sense of redundancy	Very low	Very low	—

Phase 2 — Performance

#	Item	Expected benefit	Risk	Complexity	Depends on
2.1	F-05 · debounce fast-path	–140 ms per answer; barge-in ~180 → ~30 ms	Low	Low	Scenario harness
2.2	F-08 · memoise the gate	Removes 2–63 ms of locked CPU per cycle	Very low	Very low	Gate snapshot
2.3	F-09 · cache log handles	553 → ~35 µs per event	Low	Low	Durability test
2.4	Parallelise <code>get_context</code> / <code>get_history</code>	~0 today; one round trip instead of two on Redis	Very low	Very low	—

Phase 3 — Accuracy

#	Item	Expected benefit	Risk	Complexity	Depends on
3.1	F-02 · VAD calibration + floor (together)	Recall 95.7 → 100%; false triggers 50 → 20%	Medium	Medium	Real recorded call audio; corpus re-score
3.2	Add "RAG", "vector store" and candidate names to <code>STT_PROMPT</code>	Targets the observed "hierarchical drag chart bot" class of error	Low	Very low	850-byte cap already enforced
3.3	F-04 · Unicode-aware lexical filter, or fail loudly on unsupported <code>STT_LANGUAGE</code>	Removes a total-deafness failure mode	Medium	Medium	Snapshot regression; a product decision on scope
3.4	Revisit <code>SEGMENT_END_SILENCE_MS</code> after 1.3 and 3.1	Up to –200 ms	High	Medium	Must not precede F-03

Phase 4 — Reliability

#	Item	Expected benefit	Risk	Complexity	Depends on
4.1	F-12 · client audio-device recovery	Removes a second, independent "not detected" cause	Low	Medium	Hardware testing
4.2	Observability for silent VAD rejection	Makes the hardest failure in the system visible in the logs at all	Very low	Low	—
4.3	F-10 · release per-session state	Bounded memory for a multi-user backend	Low	Low	In-flight-answer guard
4.4	F-11 · remove dead LangGraph path	One less dependency	Very low	Very low	—

Phase 5 — Advanced

#	Item	Expected benefit	Risk	Complexity	Depends on
5.1	Streaming STT (Riva adapter already exists)	Overlaps transcription with speech; could remove most of 304 ms	High	High	Infrastructure; WER comparison
5.2	Anthropic path with the existing prompt caching	Removes prompt cost from the per-minute budget entirely	Medium	Low	Quality + TTFT benchmark on <code>claude-haiku-4-5</code>
5.3	Move the gate off the pipeline-wide lock	Multi-session scaling	High	High	<code>_INTENT_MATCHERS</code> is not thread-safe
5.4	Redis backend + pub/sub answer hub	Horizontal scaling beyond one process	Medium	High	Already designed for; hub fan-out still single-process

24 · Scorecard

Area	Score		Justification
Speech detection	4		Two verified defects, both silent: deafness below −36 dBFS, 50% false-trigger on noise-only audio
STT accuracy	6		Good model and vocabulary biasing; domain terms still mangled; no WER measured; non-Latin scripts dropped
Question understanding	8		Genuinely sophisticated: clause splitting, fuzzy intent matching, focus selection, two-tier design with 97% resolved by free rules
LLM answer quality	8		92% on an independent judged rubric; format, preamble and coverage rules at 100%; spoken-voice rule weakest at ~75%
First-response latency	6		1.55 s healthy is competitive; 8–18 s when throttled, which is most of a real interview
First-audio latency	—		N/A No audio output stage exists
TTS quality	—		N/A No synthesis stage exists
Interruption handling	9		Correct cancellation, correct follow-up deferral, correct ownership re-check on a genuine race. Only bound by the debounce.
Error recovery	8		Every injected failure recovered with a typed message and no stuck state; one silent path (empty response)
Reliability / stability	6		Sound mechanics undermined by the rate ceiling: 14.7% of answers never completed corpus-wide
Resource efficiency	8		Flat across 100 interactions; one bounded cross-session retention; RSS not measured
Code quality	8		Clear separation, real vendor abstraction, careful async. Loses points for a 2,233-line module and a dead code path.
Maintainability	9		Exceptional. Nearly every constant records the measurement that produced it — the single biggest accelerator of this audit.
Observability	6		Excellent durable event log with turn ids and per-stage timings — but the most important failure (VAD rejection) emits nothing at all
Scalability	5		Single process by design; hub fan-out in memory; pipeline-wide lock holds CPU-bound work. Documented honestly by the authors.
Production readiness	6		Deployable and well-operated, but the rate ceiling makes a full interview unreliable today
OVERALL	6.9		Strong engineering held back by three specific, fixable defects

25 · Production Readiness Assessment

Verdict: not yet ready for an interview you cannot afford to lose — for reasons that are all fixable

The blocker is not architecture. It is that at **2.9 questions per minute** the application will throttle during any real interview, and when it throttles it does not fail visibly — it goes quiet for 8–18 seconds, which in a live interview is indistinguishable from being broken. Combined with a VAD that can be silently deaf to a quiet speaker, the two most likely user-visible failures both present as "it didn't respond", from completely different causes.

Ready

- Deployment shape: one Dockerfile, no vendor assumptions, env-var configuration, working health and version endpoints, token auth checked before websocket accept
- Error recovery, cancellation and barge-in semantics
- Durable per-session audit trail with turn ids and stage timings
- Vendor portability for both STT and LLM

Not ready

- Token budget vs. real question rate (F-01) — the gating issue
- Silent speech-detection failures with no telemetry (F-02, and no counter for rejected-below-trigger)
- Correctness under natural mid-sentence pauses (F-03)
- Single-process scaling limit — documented and understood, but a hard ceiling

26 · Testing Gaps

What this audit could *not* establish, stated plainly so it is not mistaken for a clean bill of health. **REQUIRES VALIDATION**

Gap	Why it matters	How to close it
No WER on real speech	STT accuracy is asserted from error <i>patterns</i> , not measured. §9 is the weakest section of this report.	Record 30–50 real interview questions with reference transcripts, including Indian-accented English, technical terms and names; compute WER per category.
Synthetic VAD audio	The absolute RMS thresholds in F-02 are calibrated against tones, not voices.	Capture loopback audio at several system volumes and speaker distances; re-derive the trigger levels before changing the threshold.
No accent / language matrix	Indian English, Hindi and code-mixed speech were tested only as text through the filter and gate — never as audio through STT.	Extend the WER dataset with Hindi and Hinglish audio once the F-04 decision is made.
No physical device testing	F-12 (device switching, Bluetooth, disconnect/reconnect) is code inspection only.	Physically disconnect devices mid-stream; test Bluetooth profile switching, docking, and default-device changes.
No process-level RSS/CPU	Internal structures are flat, but the process was never soaked under real audio.	<code>psutil</code> sampling across a 60-minute live session with real capture.
No multi-session concurrency test	The pipeline-wide lock holding 2–63 ms of CPU is a projected concern, not an observed one.	Drive 5–10 concurrent audio sockets and measure per-session latency degradation.
No unthrottled LLM baseline	Every LLM measurement was taken against an 8,000 TPM account, so healthy-path numbers are best-case snapshots between throttles.	Re-baseline on a higher tier to separate model latency from queue latency.
Quality judge unreliable on one rule	The <code>grounded</code> rule twice flagged résumé-present details as fabricated.	Majority-vote across three judge runs, or a stricter rubric with the résumé quoted inline per rule.

27 · Conclusions

The headline is that this is a well-built system with a configuration problem wearing the costume of an architecture problem. The pipeline design is genuinely good — the decoupled dispatcher/consumer, the utterance-id grouping, the three-signal end-of-speech decision, the two-tier gate that keeps 97% of questions off the network, and the barge-in semantics are all things I would expect to find in a mature product, not a first build.

What holds it back is concentrated and specific:

1. **A 1,978-token system prompt against an 8,000 TPM ceiling** puts a hard cap of ~3 questions per minute on an application whose entire purpose is answering questions faster than a human can. Every alarming number in the production logs — the 17-second p99, the 126-second maximum, the 14.7% of answers that never finished — traces back here. It is also the cheapest to fix: a benchmarked 29% token reduction with quality unchanged within measurement noise, or a provider-tier change requiring no code at all.
2. **"Sound not detected" is two bugs, not one**, and neither is the microphone. A threshold floor that defeats the adaptive logic it was meant to bound makes the app deaf below -36 dBFS; a noise estimator that cannot re-measure once it latches makes it hear speech in silence half the time. They pull in opposite directions, which is why the obvious fix — lowering the threshold — makes one of them measurably worse. There is also a third, entirely separate cause on the client (no device recovery) that would have been missed by a single-cause diagnosis.
3. **A timer is allowed to overrule the VAD**, so one spoken sentence can become two answers to two half-questions. Reproduced verbatim from a production log, and it compounds finding 1 by doubling token consumption on exactly the turns that are already struggling.

Two proposed optimisations were tested and **rejected on measurement** — audio compression for STT, and connection warm-up — and are documented as such so they are not re-proposed. That matters as much as the accepted findings: the two most obvious-looking wins in the code were not wins.

The most valuable asset for the remediation phase is the codebase's own commentary. Nearly every threshold in `config.py` records the measurement that produced it, which means every change recommended here can be evaluated against the reasoning that set the current value — not guessed at. That is unusual, and it is why this audit could reach measured conclusions rather than plausible ones.

Audit scope statement. This was a read-only audit. No application code, configuration value, prompt, model selection or threshold was modified in the delivery of this report; the working tree was verified clean against commit `7a375cf` before and after. Two configuration lines left over from an earlier exploratory session (`VAD_ENERGY_THRESHOLD`, `VAD_CALIBRATION_MS` in the gitignored `.env`) were restored to their original values and this is noted for the record. All benchmark and analysis scripts were written to a scratch directory outside the repository. Findings marked

REQUIRES VALIDATION must not be implemented without the stated tests.